

```

using System;
using System.IO;
using System.Linq;

namespace BibliotecaInteligente
{
    class Program
    {
        // =====
        // ESTRUCTURAS DE DATOS (STRUCTS)
        // =====
        public struct Libro
        {
            public stringCodigo { get; set; }
            public string Titulo { get; set; }
            public string Autor { get; set; }
            public string Editorial { get; set; }
            public int AnioPublicacion { get; set; }
            public string Categoria { get; set; }
            public int CantidadEjemplares { get; set; }
        }

        public struct Usuario
        {
            public string Carne { get; set; }
            public string NombreCompleto { get; set; }
            public string Carrera { get; set; }
            public string CorreoElectronico { get; set; }
            public string Telefono { get; set; }
            public string Estado { get; set; }
        }

        public struct Prestamo
        {
            public string CarneUsuario { get; set; }
            public stringCodigoLibro { get; set; }
            public string FechaPrestamo { get; set; }
            public string FechaEstimadaDevolucion { get; set; }
            public string Estado { get; set; }
        }

        // =====
        // ALMACENAMIENTO ESTÁTICO Y CONSTANTES
        // =====
        private const int MAX_LIBROS = 10;
        private const int MAX_USUARIOS = 5;
        private const int MAX_PRESTAMOS = 10;
    }
}

```

```

private static Libro[] _libros = new Libro[MAX_LIBROS];
private static Usuario[] _usuarios = new Usuario[MAX_USUARIOS];
private static Prestamo[] _prestamos = new Prestamo[MAX_PRESTAMOS];

private static int _contadorLibros = 0;
private static int _contadorUsuarios = 0;
private static int _contadorPrestamos = 0;

private const string DATA_DIR = "Data";
private static readonly string PathLibros = Path.Combine(DATA_DIR, "libros.csv");
private static readonly string PathUsuarios = Path.Combine(DATA_DIR, "usuarios.txt");
private static readonly string PathPrestamos = Path.Combine(DATA_DIR,
"prestamos.txt");

static void Main(string[] args)
{
    InicializarEntorno();
    EjecutarMenuPrincipal();
}

private static void InicializarEntorno()
{
    try
    {
        if (!Directory.Exists(DATA_DIR))
        {
            Directory.CreateDirectory(DATA_DIR);
        }
        CargarDatos();
    }
    catch (Exception ex)
    {
        MostrarMensajeError($"Error crítico al inicializar el sistema: {ex.Message}");
    }
}

private static void EjecutarMenuPrincipal()
{
    int opcion;
    do
    {
        Console.Clear();
        ImprimirEncabezado("SISTEMA BIBLIOTECARIO INTELIGENTE v2.0 (AI
GENERATED)");
        Console.WriteLine("1. [Módulo] Gestión de Libros");
        Console.WriteLine("2. [Módulo] Gestión de Usuarios");
        Console.WriteLine("3. [Módulo] Gestión de Préstamos");
        Console.WriteLine("4. Salir y Persistir Datos");
    }
}

```

```

Console.WriteLine("=====");
    Console.Write("Seleccione una opción: ");

    if (int.TryParse(Console.ReadLine(), out opcion))
    {
        switch (opcion)
        {
            case 1: MenuGestionLibros(); break;
            case 2: MenuGestionUsuarios(); break;
            case 3: MenuGestionPrestamos(); break;
            case 4: FinalizarSistema(); break;
            default: MostrarMensajeError("Opción fuera de rango (1-4)."); break;
        }
    }
    else
    {
        MostrarMensajeError("Entrada no válida. Ingrese un valor numérico entero.");
    }
} while (opcion != 4);
}

// =====
// SUBMENÚS MODULARES
// =====
private static void MenuGestionLibros()
{
    int op;
    do
    {
        Console.Clear();
        ImprimirEncabezado("MÓDULO A: CONTROL DE INVENTARIO Y LIBROS");
        Console.WriteLine("1. Registrar Libro");
        Console.WriteLine("2. Buscar Libro por Código");
        Console.WriteLine("3. Listar Catálogo Completo");
        Console.WriteLine("4. Eliminar Libro");
        Console.WriteLine("5. Regresar");
        Console.Write("Seleccione una opción: ");

        if (int.TryParse(Console.ReadLine(), out op))
        {
            switch (op)
            {
                case 1: RegistrarNuevoLibro(); break;
                case 2: BuscarLibroPorCodigo(); break;
                case 3: ListarCatalogoLibros(); break;
                case 4: EliminarLibroCatalogo(); break;
            }
        }
    }
}

```

```

    }
} while (op != 5);
}

```

```

private static void MenuGestionUsuarios()
{
    int op;
    do
    {
        Console.Clear();
        ImprimirEncabezado("MÓDULO B: CONTROL DE USUARIOS Y AFILIADOS");
        Console.WriteLine("1. Registrar Usuario");
        Console.WriteLine("2. Buscar Usuario (Filtro Dinámico)");
        Console.WriteLine("3. Listar Base de Usuarios");
        Console.WriteLine("4. Regresar");
        Console.Write("Seleccione una opción: ");

        if (int.TryParse(Console.ReadLine(), out op))
        {
            switch (op)
            {
                case 1: RegistrarNuevoUsuario(); break;
                case 2: BuscarUsuarioFiltrado(); break;
                case 3: ListarBaseUsuarios(); break;
            }
        }
    } while (op != 4);
}

```

```

private static void MenuGestionPrestamos()
{
    int op;
    do
    {
        Console.Clear();
        ImprimirEncabezado("MÓDULO C: GESTIÓN DE PRÉSTAMOS Y DEVOLUCIONES");
        Console.WriteLine("1. Conceder Préstamo");
        Console.WriteLine("2. Procesar Devolución");
        Console.WriteLine("3. Consultar Préstamos Activos de Usuario");
        Console.WriteLine("4. Modificar Estado del Préstamo");
        Console.WriteLine("5. Regresar");
        Console.Write("Seleccione una opción: ");

        if (int.TryParse(Console.ReadLine(), out op))
        {
            switch (op)
            {

```

```

        case 1: RegistrarTransaccionPrestamo(); break;
        case 2: RegistrarTransaccionDevolucion(); break;
        case 3: ConsultarEstadoUsuario(); break;
        case 4: ActualizarEstadoTransaccion(); break;
    }
}
} while (op != 5);
}

// =====
// LÓGICA DE NEGOCIO: MÓDULO A (LIBROS)
// =====
private static void RegistrarNuevoLibro()
{
    Console.Clear();
    ImprimirEncabezado("NUEVO REGISTRO DE LIBRO");

    if (_contadorLibros >= MAX_LIBROS)
    {
        MostrarMensajeError("Capacidad máxima de almacenamiento alcanzada para libros (Max: 10).");
        return;
    }

    string codigo = SolicitarCadenaValidada("Código Alfanumérico (8 caracteres): ", c => c.Length == 8);
    if (_libros.Take(_contadorLibros).Any(l => l.Codigo.Equals(codigo, StringComparison.OrdinalIgnoreCase)))
    {
        MostrarMensajeError("Violación de Clave Única: El código ingresado ya existe.");
        return;
    }

    Libro libro = new Libro
    {
        Codigo = codigo,
        Titulo = SolicitarCadenaValidada("Título del Libro: ", c => !string.IsNullOrEmpty(c)),
        Autor = SolicitarCadenaValidada("Autor del Libro: ", c => !string.IsNullOrEmpty(c)),
        Editorial = SolicitarCadenaValidada("Editorial: ", c => !string.IsNullOrEmpty(c)),
        AnioPublicacion = SolicitarEnteroValidado("Año de Publicación (1900 - 2026): ", 1900, 2026),
        Categoria = SolicitarCadenaValidada("Categoría: ", c => !string.IsNullOrEmpty(c)),
        CantidadEjemplares = SolicitarEnteroValidado("Cantidad de Ejemplares (Mínimo 0): ", 0, int.MaxValue)
    };

    _libros[_contadorLibros++] = libro;

```

```

        Console.WriteLine("\n[Éxito]: Entidad Libro insertada correctamente en el arreglo.");
        Console.ReadKey();
    }

    private static void BuscarLibroPorCodigo()
    {
        Console.Clear();
        ImprimirEncabezado("BÚSQUEDA INDEXADA DE LIBROS");
        Console.Write("Ingrese código de búsqueda: ");
        string target = Console.ReadLine()?.Trim();

        var libro = _libros.Take(_contadorLibros).FirstOrDefault(l => l.Codigo.Equals(target,
StringComparison.OrdinalIgnoreCase));
        if (libro.Codigo != null)
        {
            Console.WriteLine($"[Resultado Encontrado]\nCódigo: {libro.Codigo}\nTítulo:
{libro.Titulo}\nAutor: {libro.Autor}\nStock: {libro.CantidadEjemplares}");
        }
        else
        {
            Console.WriteLine("\n[Aviso]: Elemento no localizado.");
        }
        Console.ReadKey();
    }

    private static void ListarCatalogoLibros()
    {
        Console.Clear();
        ImprimirEncabezado("CATÁLOGO DE LIBROS");
        if (_contadorLibros == 0) Console.WriteLine("Matriz de datos vacía.");
        for (int i = 0; i < _contadorLibros; i++)
        {
            Console.WriteLine($"ID: {_libros[i].Codigo} | Vol: {_libros[i].Titulo} | Categ:
{_libros[i].Categoria} | Stock: {_libros[i].CantidadEjemplares}");
        }
        Console.ReadKey();
    }

    private static void EliminarLibroCatalogo()
    {
        Console.Clear();
        ImprimirEncabezado("REMOVER LIBRO");
        Console.Write("Código del libro a remover: ");
        string target = Console.ReadLine()?.Trim();

        int idx = -1;
        for (int i = 0; i < _contadorLibros; i++)
        {

```

```

        if (_libros[i].Codigo.Equals(target, StringComparison.OrdinalIgnoreCase)) { idx = i;
break; }
    }

    if (idx != -1)
    {
        for (int i = idx; i < _contadorLibros - 1; i++) _libros[i] = _libros[i + 1];
        _contadorLibros--;
        Console.WriteLine("\n[Éxito]: Estructura actualizada. Libro eliminado.");
    }
    else MostrarMensajeError("Código inexistente.");
}

// =====
// LÓGICA DE NEGOCIO: MÓDULO B (USUARIOS)
// =====
private static void RegistrarNuevoUsuario()
{
    Console.Clear();
    ImprimirEncabezado("REGISTRO DE USUARIO NUEVO");

    if (_contadorUsuarios >= MAX_USUARIOS)
    {
        MostrarMensajeError("Capacidad de memoria saturada para usuarios (Max: 5).");
        return;
    }

    string carne = SolicitarCadenaValidada("Carné Universitario (8 dígitos numéricos): ",
c => c.Length == 8 && c.All(char.IsDigit));
    if (_usuarios.Take(_contadorUsuarios).Any(u => u.Carne == carne))
    {
        MostrarMensajeError("El carné suministrado ya se encuentra indexado.");
        return;
    }

    Usuario usuario = new Usuario
    {
        Carne = carne,
        NombreCompleto = SolicitarCadenaValidada("Nombre Completo: ", c =>
!string.IsNullOrEmpty(c)),
        Carrera = SolicitarCadenaValidada("Carrera Universitaria: ", c =>
!string.IsNullOrEmpty(c)),
        CorreoElectronico = SolicitarCadenaValidada("Correo Institucional: ", c =>
c.Contains('@') && c.LastIndexOf('.') > c.IndexOf('@')),
        Telefono = SolicitarCadenaValidada("Teléfono de Contacto: ", c =>
!string.IsNullOrEmpty(c)),
        Estado = "Activo"
    };

```

```

        _usuarios[_contadorUsuarios++] = usuario;
        Console.WriteLine("\n[Éxito]: Usuario registrado y activado en la base.");
        Console.ReadKey();
    }

    private static void BuscarUsuarioFiltrado()
    {
        Console.Clear();
        ImprimirEncabezado("BÚSQUEDA FILTRADA DE USUARIOS");
        Console.Write("Ingrese término de búsqueda (Carné o Nombre): ");
        string query = Console.ReadLine()?.Trim().ToLower();

        var matches = _usuarios.Take(_contadorUsuarios)
            .Where(u => u.Carne.ToLower() == query ||
                u.NombreCompleto.ToLower().Contains(query)).ToList();

        if (matches.Any())
        {
            matches.ForEach(u => Console.WriteLine($"
-> Carné: {u.Carne} | Nombre:
{u.NombreCompleto} | Carrera: {u.Carrera} | Estado: {u.Estado}"));
        }
        else Console.WriteLine("Cero coincidencias.");
        Console.ReadKey();
    }

    private static void ListarBaseUsuarios()
    {
        Console.Clear();
        ImprimirEncabezado("REGISTROS DE USUARIOS");
        if (_contadorUsuarios == 0) Console.WriteLine("Arreglo sin registros.");
        for (int i = 0; i < _contadorUsuarios; i++)
        {
            Console.WriteLine($"
{_usuarios[i].Carne} -> {_usuarios[i].NombreCompleto}
[ {_usuarios[i].Estado}]");
        }
        Console.ReadKey();
    }

    // =====
    // LÓGICA DE NEGOCIO: MÓDULO C (PRÉSTAMOS)
    // =====
    private static void RegistrarTransaccionPrestamo()
    {
        Console.Clear();
        ImprimirEncabezado("CONCESIÓN DE PRÉSTAMO AUTOMÁTICO");

        if (_contadorPrestamos >= MAX_PRESTAMOS)

```



```

    {
        MostrarMensajeError("Límite transaccional de préstamos alcanzado (Max: 10).");
        return;
    }

    Console.Write("Ingrese Carné del Usuario: ");
    string carne = Console.ReadLine()?.Trim();
    int uldx = Array.FindIndex(_usuarios, 0, _contadorUsuarios, u => u.Carne == carne);

    if (uldx == -1 || _usuarios[uldx].Estado.Equals("Inactivo",
StringComparison.OrdinalIgnoreCase))
    {
        MostrarMensajeError("Usuario inválido o cuenta suspendida temporalmente.");
        return;
    }

    Console.Write("Ingrese Código del Libro: ");
    string codigo = Console.ReadLine()?.Trim();
    int lldx = Array.FindIndex(_libros, 0, _contadorLibros, l => l.Codigo.Equals(codigo,
StringComparison.OrdinalIgnoreCase));

    if (lldx == -1 || _libros[lldx].CantidadEjemplares <= 0)
    {
        MostrarMensajeError("Libro no disponible o sin stock físico en almacén.");
        return;
    }

    Prestamo prestamo = new Prestamo
    {
        CarneUsuario = carne,
        CodigoLibro = codigo,
        FechaPrestamo = SolicitarCadenaValidada("Fecha Préstamo (dd/mm/yyyy): ", f =>
f.Length == 10 && f[2] == '/' && f[5] == '/'),
        FechaEstimadaDevolucion = SolicitarCadenaValidada("Fecha Estimada de
Retorno (dd/mm/yyyy): ", f => f.Length == 10 && f[2] == '/' && f[5] == '/'),
        Estado = "Activo"
    };

    _libros[lldx].CantidadEjemplares--;
    _prestamos[_contadorPrestamos++] = prestamo;

    Console.WriteLine("\n[Éxito]: Préstamo procesado. Stock decrementado.");
    Console.ReadKey();
}

private static void RegistrarTransaccionDevolucion()
{
    Console.Clear();
}

```

```

        ImprimirEncabezado("RETORNO Y LIQUIDACIÓN DE LIBROS");
        Console.WriteLine("Carné del Usuario: "); string carne = Console.ReadLine()?.Trim();
        Console.WriteLine("Código del Libro: "); string codigo = Console.ReadLine()?.Trim();

        int pldx = Array.FindIndex(_prestamos, 0, _contadorPrestamos, p => p.CarneUsuario
        == carne && p.CodigoLibro.Equals(codigo, StringComparison.OrdinalIgnoreCase) &&
        p.Estado == "Activo");

        if (pldx != -1)
        {
            _prestamos[pldx].Estado = "Devuelto";
            int lldx = Array.FindIndex(_libros, 0, _contadorLibros, l => l.Codigo.Equals(codigo,
            StringComparison.OrdinalIgnoreCase));
            if (lldx != -1) _libros[lldx].CantidadEjemplares++;

            Console.WriteLine("\n[Éxito]: Libro devuelto al inventario físico.");
        }
        else MostrarMensajeError("No existe orden de préstamo activa coincidente.");
    }

    private static void ConsultarEstadoUsuario()
    {
        Console.Clear();
        ImprimirEncabezado("HISTORIAL ACTIVO");
        Console.WriteLine("Carné del estudiante: "); string carne = Console.ReadLine()?.Trim();

        var activos = _prestamos.Take(_contadorPrestamos).Where(p => p.CarneUsuario ==
        carne && p.Estado == "Activo").ToList();
        if (activos.Any()) activos.ForEach(p => Console.WriteLine($"Libro: {p.CodigoLibro} |
        Retorno: {p.FechaEstimadaDevolucion}"));
        else Console.WriteLine("Sin deudas pendientes.");
        Console.ReadKey();
    }

    private static void ActualizarEstadoTransaccion()
    {
        Console.Clear();
        ImprimirEncabezado("ACTUALIZACIÓN ESTATAL");
        Console.WriteLine("Carné del Usuario: "); string carne = Console.ReadLine()?.Trim();
        Console.WriteLine("Código del Libro: "); string codigo = Console.ReadLine()?.Trim();

        int idx = Array.FindIndex(_prestamos, 0, _contadorPrestamos, p => p.CarneUsuario
        == carne && p.CodigoLibro.Equals(codigo, StringComparison.OrdinalIgnoreCase));
        if (idx != -1)
        {
            Console.WriteLine($"Estado Actual: {_prestamos[idx].Estado}");
            Console.WriteLine("Escriba el nuevo estado (Activo/Devuelto/Vencido): ");
            _prestamos[idx].Estado = Console.ReadLine()?.Trim();
        }
    }

```

```

        Console.WriteLine("Transacción modificada con éxito.");
    }
    else MostrarMensajeError("Registro no localizado.");
}

// =====
// MANEJO DE ARCHIVOS (PERSISTENCIA AVANZADA)
// =====
private static void FinalizarSistema()
{
    try
    {
        // Serialización manual CSV - Libros
        using (var writer = new StreamWriter(PathLibros, false))
        {
            for (int i = 0; i < _contadorLibros; i++)
            {
                var l = _libros[i];

writer.WriteLine($"{l.Codigo},{l.Titulo},{l.Autor},{l.Editorial},{l.AnioPublicacion},{l.Categoria},{l.
CantidadEjemplares}");
            }
        }

        // Serialización TXT - Usuarios
        using (var writer = new StreamWriter(PathUsuarios, false))
        {
            for (int i = 0; i < _contadorUsuarios; i++)
            {
                var u = _usuarios[i];

writer.WriteLine($"{u.Carne}}{u.NombreCompleto}}{u.Carrera}}{u.CorreoElectronico}}{u.Telefo
no}}{u.Estado}");
            }
        }

        // Serialización TXT - Préstamos
        using (var writer = new StreamWriter(PathPrestamos, false))
        {
            for (int i = 0; i < _contadorPrestamos; i++)
            {
                var p = _prestamos[i];

writer.WriteLine($"{p.CarneUsuario}}{p.CodigoLibro}}{p.FechaPrestamo}}{p.FechaEstimadaD
evolucion}}{p.Estado}");
            }
        }
    }
}

```

```

        Console.WriteLine("\n[Persistencia]: Datos guardados en disco exitosamente.
Finalizando proceso.");
    }
    catch (Exception ex)
    {
        MostrarMensajeError($"Error de escritura I/O: {ex.Message}");
    }
}

private static void CargarDatos()
{
    try
    {
        if (File.Exists(PathLibros))
        {
            foreach (var linea in File.ReadAllLines(PathLibros).Where(l =>
!string.IsNullOrEmpty(l)))
            {
                var d = linea.Split(',');
                if (d.Length == 7 && _contadorLibros < MAX_LIBROS)
                {
                    _libros[_contadorLibros++] = new Libro {Codigo = d[0], Titulo = d[1], Autor
= d[2], Editorial = d[3], AnioPublicacion = int.Parse(d[4]), Categoria = d[5],
CantidadEjemplares = int.Parse(d[6]) };
                }
            }
        }

        if (File.Exists(PathUsuarios))
        {
            foreach (var linea in File.ReadAllLines(PathUsuarios).Where(l =>
!string.IsNullOrEmpty(l)))
            {
                var d = linea.Split('|');
                if (d.Length == 6 && _contadorUsuarios < MAX_USUARIOS)
                {
                    _usuarios[_contadorUsuarios++] = new Usuario { Carne = d[0],
NombreCompleto = d[1], Carrera = d[2], CorreoElectronico = d[3], Telefono = d[4], Estado =
d[5] };
                }
            }
        }

        if (File.Exists(PathPrestamos))
        {
            foreach (var linea in File.ReadAllLines(PathPrestamos).Where(l =>
!string.IsNullOrEmpty(l)))
            {

```

```

        var d = linea.Split('|');
        if (d.Length == 5 && _contadorPrestamos < MAX_PRESTAMOS)
        {
            _prestamos[_contadorPrestamos++] = new Prestamo { CarneUsuario =
d[0], CodigoLibro = d[1], FechaPrestamo = d[2], FechaEstimadaDevolucion = d[3], Estado =
d[4] };
        }
    }
}
catch
{ // Manejo silencioso en archivos vacíos o corruptos iniciales
}
}

// =====
// COMPONENTES AUXILIARES DE COGNICIÓN/VALIDACIÓN (HELPERS)
// =====
private static string SolicitarCadenaValidada(string etiqueta, Func<string, bool> criterio)
{
    while (true)
    {
        Console.Write(etiqueta);
        string input = Console.ReadLine()?.Trim();
        if (criterio(input)) return input;
        Console.WriteLine("[Error]: Restricción de formato violada. Reintente.");
    }
}

private static int SolicitarEnteroValidado(string etiqueta, int min, int max)
{
    while (true)
    {
        Console.Write(etiqueta);
        if (int.TryParse(Console.ReadLine(), out int valor) && valor >= min && valor <=
max) return valor;
        Console.WriteLine($"[Error]: El valor debe ser un entero entre {min} y {max}.");
    }
}

private static void ImprimirEncabezado(string titulo)
{
    Console.WriteLine("=====");
    Console.WriteLine($" {titulo}");

    Console.WriteLine("=====");
}

```

```
private static void MostrarMensajeError(string m)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine($"\\n[ERROR]: {m}");
    Console.ResetColor();
    Console.ReadKey();
}
}
}
```